1.8 Asymptotic Notations

Big-O, Big-Ω and Big-Θ — Abdul Bari Lecture Notes

1.8.1 Big-O, Big-Ω and Big-Θ

Introduction: Asymptotic notations originate from mathematics. They represent the growth of a function and are commonly used to express the time complexity of algorithms. They provide a concise way to represent growth instead of writing the complete function.

Notation	Meaning	Memory
Big-O (O)	Upper Bound	At most
Big-Omega (Ω)	Lower Bound	At least
Big-Theta (Θ)	Tight Bound	Both / tight

2. Big-O Notation — O (Upper Bound)

Definition: f(n) = O(g(n)) if there exist positive constants c and n■ such that:

$$\mathbf{f(n) \le c \cdot g(n), \quad for\ all\ n \ge n■}$$

Example: f(n) = 2n + 3. For n ≥ 1: 2n + 3 ≤ 2n + 3n = 5n. Hence **f(n) = O(n)**, where c = 5 and g(n) = n.

Important: O(n²) and O(2■) are also mathematically valid upper bounds for 2n+3, but are too loose. Choose the closest useful upper bound, O(n). O(log n) cannot be an upper bound here.

3. Big-Omega Notation — Ω (Lower Bound)

Definition: f(n) = Ω(g(n)) if there exist positive constants c and n■ such that:

$$\mathbf{f(n) \ge c \cdot g(n), \quad for\ all\ n \ge n■}$$

Example: f(n) = 2n + 3. For n ≥ 1: 2n + 3 ≥ n. Hence **f(n) = Ω(n)**, where c = 1 and g(n) = n.

Important: Ω(log n) is also mathematically valid, but Ω(n) is the closest useful lower bound. Ω(n²) cannot be a lower bound for 2n+3.

4. Big-Theta Notation — Θ (Tight Bound)

Definition: f(n) = Θ(g(n)) if there exist positive constants c■, c■ and n■ such that:

$$\mathbf{c■g(n) \le f(n) \le c■g(n), \quad for\ all\ n \ge n■}$$

Example: For f(n) = 2n + 3: n ≤ 2n + 3 ≤ 5n. Therefore **f(n) = Θ(n)**. Θ is preferred whenever possible because it gives the tight rate of growth.

5. Bounds vs. Cases

Algorithmic cases: Best Case, Worst Case and Average Case describe different input situations. **Asymptotic bounds:** O = upper bound, Ω = lower bound, Θ = tight bound. Therefore, Big-O does not automatically mean worst case, and Big-Omega does not automatically mean best case. Best, worst and average cases can each be described using asymptotic notations.

1.8.2 Solved Examples

Example 1 — Polynomial Function

Given: **f(n) = 2n² + 3n + 4**

Upper: For n ≥ 1, 3n ≤ 3n² and 4 ≤ 4n². Thus 2n² + 3n + 4 ≤ 9n² ⇒ **O(n²)**.

Lower: 2n² + 3n + 4 ≥ n² ⇒ **Ω(n²)**.

Tight: n² ≤ 2n² + 3n + 4 ≤ 9n² ⇒ **Θ(n²)**.

Example 2 — Log-Linear Combination

Given: **f(n) = n² log n + n**. Dominant term = n² log n.

Upper: n² log n + n ≤ 10n² log n ⇒ **O(n² log n)**.

Lower: n² log n + n ≥ n² log n ⇒ **Ω(n² log n)**.

Therefore: **Θ(n² log n)**. Growth: **n² < n² log n < n³**.

Example 3 — Factorial Function

n! = 1 × 2 × 3 × ⋯ × n. Every factor ≤ n, so n! ≤ n■ ⇒ **O(n■)**. Every factor ≥ 1, so n! ≥ 1 ⇒ **Ω(1)**. These simple bounds do not match, so this method does not directly give a useful Θ bound.

Example 4 — Log Factorial

log(n!) = log1 + log2 + ⋯ + log n. Since every factor ≤ n: log(n!) ≤ log(n■) = n log n ⇒ **O(n log n)**. The simple lower argument gives log(n!) ≥ log1 = 0, a trivial lower bound.

6. Important Takeaways

• **O → Upper Bound** • **Ω → Lower Bound** • **Θ → Tight Bound**

Choose the closest/tightest useful bound. For example, 2n+3 = O(n) is preferred over O(n²), although both are mathematically true.

Phone Price Analogy

If a phone costs approximately ■20,000: Θ ≈ ■20,000 (tight estimate); Ω = at least ■2,000 (lower bound); **O** = at most ■2,00,000 (upper bound). The last two are technically correct but too broad.

■ Quick Memory

O → Upper | Ω → Lower | Θ → Tight

Best/Worst/Average Case ≠ O/Ω/Θ